



ISIM — Fastlane Engine

Manuel d'utilisation



2026

Rédigé par

Lucas ESTRADE & César CAPION

Table des matières

1 *Introduction* **2**

2 *Présentation & Instructions* **2**

3 *Commandes* **3**

4 *Bibliographie* **3**

1. *Introduction*

Fastlane Engine est un projet de synthèse d'images. C'est un moteur qui offre plusieurs méthodes de calcul pour générer une route dans un type de rendu que l'on appelle *2.5D*, ou *pseudo 3D*. Ces méthodes de rendu étaient très populaires pour les jeux de course à l'époque des consoles 8 bits et 16 bits, car ces unités de calcul n'étaient pas assez puissantes pour générer une scène en trois dimensions en temps réel.

2. *Présentation & Instructions*

Le moteur contient plusieurs méthodes de calcul, chacune étant associée à une fonction définie dans la classe `Engine`. Pour lancer le projet, il suffit de lancer la commande `make` dans son terminal, tout en s'assurant que c'est bien la fonction `projection22()` qui est utilisée dans le fichier *main.cpp*.

Le tracé de la route, les zones de sprite, ainsi que toutes les textures sont personnalisables, les classes contiennent des fonctions de chargement. Les fichiers de route se configurent de cette façon : il suffit d'écrire <*altitude*> <*centerOffset*> sur plusieurs lignes, puis le moteur se charge d'interpoler proprement les valeurs grâce à la méthode de Catmull-Rom. En cas de doutes, le fichier *roads/track.txt* peut servir d'exemple. Certaines textures sont chargées et affichées avec de la transparence. Pour rendre un pixel transparent dans une texture, il suffit de lui appliquer la couleur magenta.

La fonction `projection22()` constitue la méthode de calcul la plus avancée du projet, elle utilise la méthode projective, c'est à dire qu'elle parcourt le monde (la route/le sol) puis elle peint une ou plusieurs lignes de l'écran par interpolation. Elle est définie dans le fichier *engine.cpp*.

Un second fichier appelé *engineLegacy.cpp* définit toutes les méthodes obsolètes. La méthode `projection2()` est une version plus ancienne de la fonction précédente, surtout au niveau de sa structure, même si elle comporte quasiment les mêmes fonctionnalités que `projection22()`. Quelques parties du code sont commentés dans cette fonction, pour exposer les anciennes pistes de réflexion, comme par exemple pour le calcul de l'altitude.

La méthode `scanline()` est différente des deux autres, elle fonctionne avec la même formule mathématique, sauf que la pipeline est inversée : on parcourt chaque ligne de l'écran puis on détermine la profondeur de la route pour la tracer. Elle offre de bonnes performances mais est malheureusement très limitée lorsqu'il s'agit de déformer la route.

La méthode `basic()` est expérimentale et ne s'appuie sur aucune formule mathématique. Elle ne fait que calculer des bandes de couleur.

Pour compiler ces trois méthodes obsolètes, il faut taper la commande `make legacy`, et il est ensuite possible de changer la méthode utilisée dans *main.cpp*. Voici un tableau récapitulatif des méthodes.

Fonctionnalité	`basic()`	`scanline()`	`projection2()`	`projection22()`
Texture route	×	✓	✓	✓
Décorations	×	✓	✓	✓
Virages	×	×	✓	✓
Relief	×	×	✓	✓
Texture herbe	×	×	✓	✓
Ciel (parallax)	×	×	~	✓
Commandes camera	×	~	~	✓
Segmentation (objet)	×	×	×	✓

3. *Commandes*

+ / −	⟶	Régler la vitesse
Z* / *S* / *Q* / *D	⟶	Déplacer la caméra
◄ / ► / ▲ / ▼	⟶	Pivoter la caméra
↩ / ←	⟶	Régler la distance focale
Y* / *H	⟶	Régler la courbure de la route
B* / *N	⟶	Régler le relief de la route
T* / *G	⟶	Régler simultanément l'horizon et la hauteur de la caméra
O* / *L	⟶	Régler la force d'inclinaison verticale de la caméra
U* / *J	⟶	Régler la force d'inclinaison horizontale de la caméra
R	⟶	Activer / Désactiver le suivi de l'orientation
F	⟶	Activer / Désactiver le suivi latéral
V	⟶	Activer / Désactiver le suivi de hauteur
C	⟶	Réinitialiser tous les réglages

4. *Bibliographie*

Lou's Pseudo 3D Page